

# My top 9 fav Crypto's

Not Cryptocurrency, fav Cryptography

- By Tom Cope (<https://tomcope.com>) [2026-06-21]

Warning: This presentation was entirely human generated and may contain opinions



# TEA - Tiny Encryption Algorithm



- It's so simple and cool!
- What a fantastic name!
- Not very secure
  - 1 Key = 3 Keys
  - Was used to break the original XBOX
- Used in Valve Anti Cheat
  - Encrypts the different scanning modules
- Still fairly used due to its simplicity
- Tea = 1994 (Roger Needham, David Wheeler)
- XTea = 1997 (Roger Needham, David Wheeler) (A bit more secure)
- XXTea = 1998 (Roger Needham, David Wheeler) (Even more a bit more secure!)

```
#include <stdint.h>

void encrypt (uint32_t v[2], const uint32_t k[4]) {
    uint32_t v0=v[0], v1=v[1], sum=0, i;           /* set up */
    uint32_t delta=0x9E3779B9;                      /* a key */
    schedule_constant */
    uint32_t k0=k[0], k1=k[1], k2=k[2], k3=k[3];    /* cache key */
    for (i=0; i<32; i++) {                          /* basic cycle
start */
        sum += delta;
        v0 += ((v1<<4) + k0) ^ (v1 + sum) ^ ((v1>>5) + k1);
        v1 += ((v0<<4) + k2) ^ (v0 + sum) ^ ((v0>>5) + k3);
    }
    v[0]=v0; v[1]=v1;
}
```

# OTR - Off the Record



- “What if we had a system which could run on top of ANY messaging platform in order to make it secure”?
  - ?OTR?v23?
  - ?OTR:AAQD9P....
- Invented in 2004 by Nikita Borisov, Ian Goldberg, and Eric A. Brewer.
- Supports 4 strong guarantees
  - Encryption
  - Authentication
  - Perfect Forward Secrecy (PFS)
  - Plausible Deniability (ensuring that no third party could mathematically prove that a specific person wrote a particular message)
- Natively supported in Go!
  - <https://pkg.go.dev/golang.org/x/crypto/otr>



# OTR - Off the Record



- Supports 4 strong guarantees:
  - Authentication
    - How do you know the person you are talking to on the other end is the right person?
      - Out of bound checks (Call them up and check fingerprints)
      - What if you could ask them a question?
        - “what was the name of my old cat”

# OTR - Off the Record

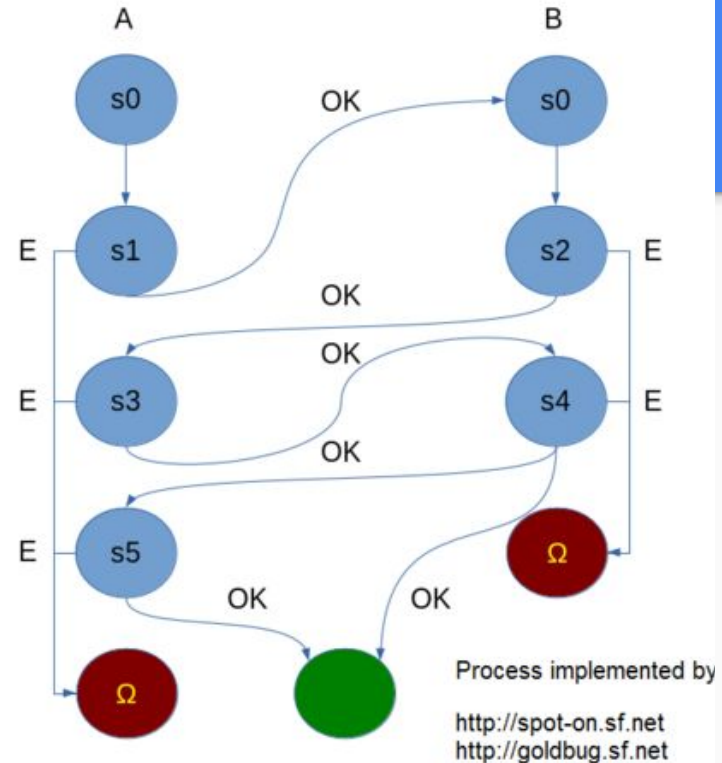
In [cryptography](#), the **socialist millionaire problem**<sup>[1]</sup> is one in which two millionaires want to determine if their wealth is equal without disclosing any information about their riches to each other.

OTR uses this to ask a question and if you get it right then you establish a secure connection

- “What was the name of my old cat”
- “Tabby”
- <Connection Authenticated, Verified and Secure>

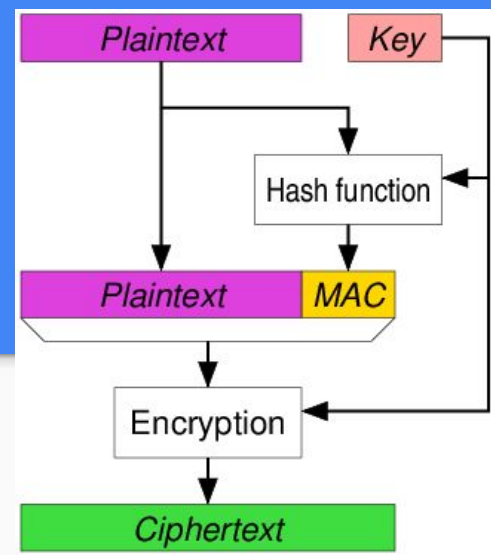
## SMP - Socialist Millionaire Protocol

State Machine



# ChaCha20-Poly1305

- AES (Advanced Encryption Standard)
  - “cheats” -> Hardware black box acceleration
  - Special intel instruction (AES-NI)
- Can't check inside the CPU Blackbox for the implementation
- ChaCha20-Poly1305, is a great alternative which used Authenticated Encryption with Associated Data (AEAD)
  - Created by Daniel J. Bernstein in 2005(ChaCha) & 2008 (Poly)
- Its really fast and very secure!
- Used in many mobile and low powered devices
- Picked up by Google in order to make phones better at using the internet!
- Used in TLS and SSH
- It's the “cool” competitor, good to not use one crypto for everything





# lib sodium

- Not really a crypto algorithm
- A Cool library for making stuff with crypto
- Designed to protect you from mistakes
- “Sodium is a modern, easy-to-use software library for encryption, decryption, signatures, password hashing, and more. It is a portable, cross-compilable, installable, and packageable fork of NaCl, with a compatible but extended API to improve usability even further”



# libsodium

## Example (client-side)

```
unsigned char client_pk[crypto_kx_PUBLICKEYBYTES], client_sk[crypto_kx_SECRETKEYBYTES];
unsigned char client_rx[crypto_kx_SESSIONKEYBYTES], client_tx[crypto_kx_SESSIONKEYBYTES];

/* Generate the client's key pair */
crypto_kx_keypair(client_pk, client_sk);

/* Prerequisite after this point: the server's public key must be known by the client */

/* Compute two shared keys using the server's public key and the client's secret key.
   client_rx will be used by the client to receive data from the server,
   client_tx will be used by the client to send data to the server. */
if (crypto_kx_client_session_keys(client_rx, client_tx,
                                  client_pk, client_sk, server_pk) != 0) {
    /* Suspicious server public key, bail out */
}
```

## Example (server-side)

```
unsigned char server_pk[crypto_kx_PUBLICKEYBYTES], server_sk[crypto_kx_SECRETKEYBYTES];
unsigned char server_rx[crypto_kx_SESSIONKEYBYTES], server_tx[crypto_kx_SESSIONKEYBYTES];

/* Generate the server's key pair */
crypto_kx_keypair(server_pk, server_sk);

/* Prerequisite after this point: the client's public key must be known by the server */

/* Compute two shared keys using the client's public key and the server's secret key.
   server_rx will be used by the server to receive data from the client,
   server_tx will be used by the server to send data to the client. */
if (crypto_kx_server_session_keys(server_rx, server_tx,
                                  server_pk, server_sk, client_pk) != 0) {
    /* Suspicious client public key, bail out */
}
```



# libsodium

## Secret Box

### Example

```
#define MESSAGE (const unsigned char *) "Message"
#define MESSAGE_LEN 7
#define CIPHERTEXT_LEN (crypto_box_SEALBYTES + MESSAGE_LEN)

/* Recipient creates a long-term key pair */
unsigned char recipient_pk[crypto_box_PUBLICKEYBYTES];
unsigned char recipient_sk[crypto_box_SECRETKEYBYTES];
crypto_box_keypair(recipient_pk, recipient_sk);

/* Anonymous sender encrypts a message using an ephemeral key pair
 * and the recipient's public key */
unsigned char ciphertext[CIPHERTEXT_LEN];
crypto_box_seal(ciphertext, MESSAGE, MESSAGE_LEN, recipient_pk);

/* Recipient decrypts the ciphertext */
unsigned char decrypted[MESSAGE_LEN];
if (crypto_box_seal_open(decrypted, ciphertext, CIPHERTEXT_LEN,
    recipient_pk, recipient_sk) != 0) {
    /* message corrupted or not intended for this recipient */
}
```

<https://pkg.go.dev/golang.org/x/crypto/nacl/secretbox>

```
func Open(out, box []byte, nonce *[24]byte, key *[32]byte) ([]byte, bool)
func Seal(out, message []byte, nonce *[24]byte, key *[32]byte) []byte
```

## Uses ChaCha20-Poly1305!

# Signal - Sealed Sender



- Signal app goes a very long way to protect users with both encryption and data minimization
  - Less meta data collection better
  - Unlike WhatsApp which keeps your social graph

When you send a message you need:

- Sender
- Receiver
- >What if we could change that?<

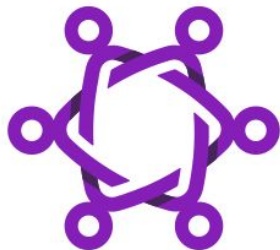


# Signal - Sealed Sender



# MLS - Message Layer Security

- The Messaging Layer Security (MLS) Protocol RFC 9420
- The “Signal Competitor”
- Difference
  - Signal = Designed for 1-2-1 Chats & Small groups
  - MTLS = Designed for MASSIVE groups (2 -> 50,000)
- Designed based on Asynchronous Ratcheting Trees



**MLS**

# Ed25519

- They are so small!!!!!!!
- Same Security as RSA but used Elliptic Curves

# RSA (2048 bits) Vs Ed25519 (256 bits)

-----BEGIN PRIVATE KEY-----

MIIEvAIBADANBgkqhkiG9w0BAQEFAASCbKYwggSiAgEAAoIBAQCaq6y27+AcrHGP  
0lQO4Ckf/1IXJ3J5H0e5R/ttltOUOgnfSjHbK3N/gDBVK5ruV1kAzLP1RrLuHqS2  
ranU2NFWwqFB87tyfu92os8qQP4kC4a4QUL3CH8vMMNIbKxFNUP+R/4BwrMtG6t8  
B2UxPmfo2E4KfdwzZElc.JsH4vYOYure9KV3JsXO/CfrIzvg6+Zmw7sCGwFSsbRkp  
enqYv3qmEKUfF9stHDlB0ER6mBmU610EuTqqqbzpLJ6tBQxHDuL5hrc1hsDLNyTI  
Nsxst2yXHRk0LvZnYOjeP5/Yu0aB5qGg+FwNVWral0LBzNn48sdqTifg9lJk2m7D  
TilvVlnJAgMBAECCggEACfe5VDRGPYLP2LxtxV2l2NWMbOTYacLfbO3alrhfH0TU  
QQsXhMLnmRZqRvrd67E+X6JRkKODQXxBEjLD6J8mcohtewlZmOncRdF9N1Rj7G+/  
uV0fUys9gw2J8D9spLMnlBoQoQ1qnaQUW5RgBZl/ec6CzQNKbx4htiobQww3kJ6S  
KsKMxeqcCOtuA7WgQsYOGiOMHLOApMID5cigP25ZdvMu9dmQ+iDhJgP/DbAbsZOW  
TguBcJeMxNGlhmDgywpVbA3Xf1ecMv4p8H2rZbsZEBj9TjsUWWqVrtmUeiWVCbZA  
AZLbuA0HuTjgrgmIZUa6egW45UsObqNKO7434zs2PwKBgQDS9xsKTCQdfZz2JAIt  
utparQ0yrnutEb8vC+KWP7hgDhAUF66552EXlIt0XeGHPY+teadlcuOeG4mtlPDu  
kFg0lb5y2BEJJAhYW7VKOmLPfKsnU8YSjBEvQST7ETMx2VPS45/U25+6JOjsXaAQ  
4j8lUy76CMpXxqDDMewRGJJkwKBgQC7sClaGXNlvZBcEd3wjldTnKXN9WLJ9FT0  
FWD5d9A5lEu7fltLx+BVXOHn7Afr+h4p5jpx1igrsQg824xkg9lZr2Gdi9pNc9JG  
5+eHBXodj6r83VNOKQiuKWhkABxE1AQFFw/GRIubqOrRn4aeT0mnj4SXCapyCqYd  
dUqxQ2B4swKBgGns3Mzc0j1kds035uwLp9tjjH7oEP0jLgVnSEv8jozcQsJz6a06  
OgussZiqErUe+ec2cBCGETevx/7/zTlBj15KqadsSZF3YCFNwsd8bWSUyjMi4w8o  
+O/rG7BcQlJFBsuaswPjil2ibrPgTE5CE+rZqjwhPhxqQKZLFihD6i1AoGAJiHV  
VPPkyR/pa3tJzyyN9gPWf/DCAGv+spSBXk3kZT5m4JGeL2hNgKpdT2RZ+6yWYkbf  
Su+Ku8288bhdXrKVi2CvgXW/hYrphvRoyrrn14hj4foJieXXkKLjDaYkGCvHIU2d  
K9gTUS3Rf4Ncs17G/f8fnhwdmsOXGlcRNNOckhCgYAVRTK/VSFu4VvJIAZxQZsN  
tUFnCM3uo53elEfzdXCRI+Div6laNPlvKHZ9nqVXbhfuneibXxglpCxHVGARftit  
R5vxVjzRxskbFr1kmwybpYbN8wcDJVqCLMhCzgaiahZOu8ux3Qf34aqXcj5C63mi  
VghnhzXhwjrfnwTCOXnstQ==

-----END PRIVATE KEY-----

-----BEGIN OPENSSH PRIVATE KEY-----

b3BlbnNaZC1rZXktZjEAAAAAAAAABG5vbmUAAAAAAAAABAAAAEbm9uZQAAAAAAAAABAAAAA  
AAAAAAAtzc2gtZWQyNTUxOQAAAAAAAAACAts8f9ONDMtzHh7MptEcjbGrjnXH55HFNZPudNN41y7wAAALiEyd1hMnb  
dQAAAAAtzc2gtZWQyNTUxOQAAAAAAAAACAts8f9ONDMtzHh7MptEcjbGrjnXH55HFNZPudNN41y7w  
AAAAEA0Cvyv8efJn8kdwSstl/XSbU8LaVz3cqjfGCSRUstXli2zx/040My3MeHsym0RxuMa  
uOdcfnkcU1k+5003jXLvAAAAABHRlc3QB

-----END OPENSSH PRIVATE KEY-----

## SafeCurves

[illegible]

# AGE



- Created by Filippo Valsorda (The Go Crypto Core Maintainer)
- “age is a simple, modern and secure file encryption tool, format, and Go library”
- “It features small explicit keys, post-quantum support, no config options, and UNIX-style composability”
- Uses both:
  - Ed25519
  - ChaCha20-Poly130
- Really great way of encrypting data and sharing it with people
  - <https://tomcope.com/agepubkey>
  - Age1g94elu386ghaysp6x947ze2snaz2p569nhhwlp7m5e75jgedfusgcc4qp
  - Supports password encryption too!

# Cryptomator



- Allows you to create encrypted folders on your PC
- Acts as middle box between your local PC and the Cloud
- Open Source and Good Crypto
- Really easy to use
- Free and has a great mobile app (with share function)
- Designed to work nicely with sync and backup services

Thank you